
UNIVERSITY OF PISA

DEPARTMENT OF COMPUTER SCIENCE

Master's Degree in Computer Science

# AMM Scheduler:

## Design and Implementation of a Runtime Scheduling System for Asymmetric Heterogeneous Computing

Supervisor:

**Prof. Marco Danelutto**

Candidate:

**Simone Stanganini**

ACADEMIC YEAR 2024/2025

# Contents

# Chapter 1

# Introduction

## 1.1 Contest

Until a few years ago, performance improvements depended mainly on increasing CPU clock speeds, today the approach has changed completely, shifting towards hardware specialization.

Modern computers, from servers to embedded SoCs, don't rely on a single processor anymore, instead they integrate different specific accelerators:

- **CPU**: Optimized for sequential tasks and low latency.
- **GPU**: Available as integrated (iGPU) or discrete (dGPU), in a lot of cases both at the same time, designed for massive parallel computing.
- **NPU** (Neural Processing Unit): A recently introduced unit, specialized in speeding up matrix operations for AI workloads.

This variety of hardware has created a gap with the software, while hardware offers different resources for different workloads, traditional software tends to be static. It usually assigns tasks to a single device ,often just the CPU or GPU, and ignores the other available resources.

The result is an inefficient use of the system. We could get more throughput with the same efficiency, or be more efficient at the same speed, so the current challenge is *orchestration*: we need a system that can dynamically map each task to the right accelerator, using the entire heterogeneous topology of the system.

## 1.2 Objectives

The main goal of this thesis was to design and implement a scheduler capable of dynamically assigning computational tasks, specifically streams of matrix multiplications, to the different accelerators found in modern architectures.

The work started with an essential preliminary phase, I focused on integrating and studying different compute backends, such as CUDA, SYCL, OpenVINO. This step was useful to create a uniform abstraction layer over the hardware and to verify the technical feasibility of the system.

After this initial phase, the work moved on to the actual development of the scheduler, the core of this project focused on the asymmetric orchestration logic, designed to decide in real-time which resource to use for each specific workload for maximize utilization and efficiency.

Finally, the system was validated with experimental results on a complex hybrid architecture, this setup included an Intel multicore CPU (with integrated GPU and NPU) and a discrete NVIDIA GPU. The tests used to verify the operation were streams of identical matrices, streams of heterogeneous matrices, and splitting larger tasks across multiple accelerators.

## 1.3 Thesis Structure

The work is organized into the following chapters:

- **Chapter 2**: Introduces the hardware technologies (CPU, GPU, NPU), the software backends used (CUDA, SYCL, OpenVINO, OpenBLAS) and the software stack used for this project (C, C++, Cmake).
- **Chapter 3**: Describes the logical design, divided into two phases: the creation of the benchmarking environment and the subsequent design of the scheduler.
- **Chapter 4**: Details the code implementation, it covers the wrappers for hardware abstraction, the essential data structures for the scheduler (ringbuffer and performance map), and the techniques used for task splitting, as well as the project compilation procedures and details.
- **Chapter 5**: Analyzes the experimental results, validating the effectiveness of the scheduler on different workloads.
- **Chapter 6**: Draws conclusions on the work done and suggests possible future developments.

# Chapter 2

# Tools and Technologies

This chapter presents the hardware and software resources used to develop and validate the system. The choice to work on a hybrid and highly heterogeneous architecture was made to test the scheduler in a real-world scenario, and in this scenario, devices with different performance characteristics and programming models have to work together.

## 2.1 Experimental Platform

All development and benchmarking activities were done on a mobile workstation Lenovo Yoga Pro 9i gen 9. This platform represents a modern high-performance "Edge" computing node well, it integrates all the types of accelerators studied into a single physical system: multicore CPU, NPU, integrated GPU, and discrete GPU, and having these components available at the same time allowed to emulate, within a single node, the orchestration issues of heterogeneous distributed systems.

### 2.1.1 The SoC: Intel Core Ultra 9 185H

The core of the system is the Intel Core Ultra 9 185H processor, based on the "Meteor Lake" architecture [2]. This component marks a significant change from traditional monolithic CPUs. It uses a disaggregated design (tiled architecture) that combines different specialized processing units:


Figure 2.1: Intel Ultra Logo

- **Host CPU**: This cpu is made of 16 hybrid cores, 6 Performance-cores, 8 Efficient-cores, and 2 Low-Power-cores for efficiency. In this project this unit acts both as host and accelerator, it runs the scheduler code and does some computation through the OpenBlass library (2.5).
- **NPU**: The Ultra 9 includes a native Neural Processing Unit. This is a low-power accelerator designed specifically for neural network inference. Its compute acceleration is facilitated by Neural Compute Engines, which consist of hardware acceleration blocks for AI operations like Matrix Multiplication and Convolution, alongside Streaming Hybrid Architecture Vector Engines for general computing tasks [4]. In this project the NPU is used via the OpenVINO 2.4 backend to run Matrix Multiplication.
- **iGPU**: Intel Arc Graphics, integrated into the SoC, this graphics unit based on Xe-LPG architecture [1], offers parallel computing capabilities with direct access to system memory, in this project this accellerator has been used with the SYCL library (2.3) as the backend to leverage the full performance of this accellerator.

### 2.1.2 NVIDIA GeForce RTX 4060


Figure 2.2: Nvidia Rtx Logo

To complement the Intel SoC the system is equipped with a discrete NVIDIA GeForce RTX 4060 Laptop GPU. This device is based on the Ada Lovelace architecture [8] and acts as the high-throughput accelerator of the system.

Unlike the integrated GPU this card features 3072 CUDA Cores for general parallel processing and 96 Fourth-Generation Tensor Cores. The Tensor Cores are specifically designed to accelerate dense matrix multiplications with 16 or even 8 bit operations.

The device utilizes 8 GB of GDDR6 VRAM with a 128-bit memory interface, this dedicated memory provides high bandwidth but requires explicit data transfer over the PCIe bus.

Within the scheduler this device is managed via the CUDA backend 2.2.

## 2.2 CUDA Toolkit


Figure 2.3: Nvidia Cuda Logo

To exploit the computational capabilities of the discrete NVIDIA GPU the system utilizes CUDA (Compute Unified Device Architecture). This parallel computing platform and programming model developed by NVIDIA that enables dramatic increases in computing performance by harnessing the power of the GPU. It allows developers to accelerate compute- intensive applications using C, C++, and Fortran, and is widely adopted in fields such as deep learning, scientific computing, and high-performance computing (HPC) [10].

In this project CUDA is the tool chosen to extract maximum throughput from the hardware 2.1.2. The implementation relies on specific libraries and mechanisms to optimize the execution of matrix multiplication streams.

- **cuBLAS and Tensor Cores**: to perform linear algebra operations we adopts cuBLAS (CUDA Basic Linear Algebra Subroutines). This library is the GPU-accelerated version of the standard BLAS specification and provides highly optimized implementations of matrix operations, specifically, cuBLAS allows the utilization of Tensor Cores, specialized hardware units designed to accelerate mixed-precision matrix multiplication (Float16 and Int8). This significantly increases the theoretical throughput compared to standard floating-point units [9].

- **CUDA Streams**: To handle the latency introduced by data transfers between the CPU and the GPU the system utilizes CUDA Streams. A stream is a sequence of operations that execute in issue-order on the GPU, by employing multiple streams it is possible to achieve concurrency: while one stream is transferring data over the PCIe bus another stream can execute a computational kernel [13]. This mechanism, known as "latency hiding", is fundamental for real-time processing and ensures that the GPU compute units remain active as much as possible.

- **NVIDIA Management Library (NVML)**: For the analysis of energy efficiency the project integrates the NVIDIA Management Library (NVML). This serves as a monitoring tool that provides direct access to the GPU hardware sensors, it allows the retrieval of real-time metrics such as power usage (in milliJoule) and temperature without requiring external physical probes [11].

## 2.3 SYCL

While CUDA provides maximum performance for NVIDIA hardware, it requires a proprietary programming model, and to address the need for portability and to fully exploit the Intel hardware present in the system, the project adopts the SYCL standard through the Intel oneAPI toolkit.

- **The SYCL Standard**:


Figure 2.4: SYCL Logo

SYCL is an open, royalty-free standard managed by the Khronos Group. It enables high-performance computing on heterogeneous accelerators (such as CPUs, GPUs, and FPGAs) using standard ISO C++ [15]. As defined in the official specification, SYCL provides an abstraction layer that allows developers to write code for heterogeneous processors using a "single-source" style. This means that host code on the CPU and the device code on the accelerator are written in the same file, utilizing standard C++ templates and lambda functions. This approach eliminates the complexity of managing separate source files for different architectures.

- **Intel oneAPI**:


Figure 2.5: OneApi Logo

SYCL requires a specific software development kit (SDK) to be compiled and executed. For this thesis, the Intel oneAPI Base Toolkit was selected [3]. oneAPI is Intel's unified programming model designed to simplify development across diverse architectures, it implements the SYCL standard via the `icpx` compiler, and provides a comprehensive set of libraries optimized for a large set of devices. The choice of this toolkit allows the project to target the available Intel hardware efficiently. Moreover this choice guarantees that the codebase could seamlessly target accellerator from othe vendor like AMD without having to rewrite the code.

## 2.4 Openvino


Figure 2.6: OpenVino Logo

Developed by Intel, this open-source toolkit is designed to optimize and deploy deep learning models across various types of hardware, from edge devices to cloud servers [5].

The main function of the library is that it acts as a translator, it takes an AI model and converts it into a specific format that the target hardware understands. This is very useful because it allow the program to run on different components like the CPU, NPU or the integrated graphics without needing to rewrite the code for each specific accellerator.

The toolkit automatically handles the low-level details to ensure the task runs as fast as possible in the chosen device. This efficiency is achieved through the model optimizer, as described in the documentation, this tool analyzes the computational operations and simplifies them (for example, by merging multiple mathematical steps into one) before execution.

The result is a hardware-agnostic format called intermediate representation, IR, which allows the runtme to map the operations efficiently to the available resources [5].

In this project OpenVINO was essential for targeting the specific NPU found in the system 2.1.1. Since the NPU cannot run standard C++ code directly, OpenVINO acts as the bridge between the scheduler and the hardware, specifically, it was implemented to execute Matrix Multiplication tasks on the NPU. This allowed the system to offload these calculations to the neural accelerator, taking advantage of its high energy efficiency.

## 2.5 OpenBlass


Figure 2.7: OpenBlass Logo

To enable efficient processing on the Host CPU, the project integrates OpenBLAS. OpenBLAS is an optimized, open-source implementation of the BLAS (Basic Linear Algebra Subprograms) API.

Historically, the project emerged as a successor to GotoBLAS2, a highly regarded library developed by Kazushige Goto, then after the original development ceased, the OpenBLAS community took over to maintain the code and expand support for modern architectures [16].

The library achieves high performance through a mechanism called runtime processor detection. OpenBLAS first identifies the CPU architecture and automatically selects the most optimized functions for that specific hardware, these functions are often implemented in assembly language to leverage SIMD instructions, such as AVX2 or AVX-512, which allow the processor to handle multiple data points simultaneously. Additionally, the library automatically handles multi-threading to distribute the computational load across all available processor cores[12].

In this project this library allow the Host CPU to be treated as an additional accelerator, so rather than limiting the CPU solely to orchestration tasks, OpenBLAS enables the scheduler to use some of the processor cores as a worker node. More details on this will be provided in the Implementation Chapter 4.

## 2.6 C/C++

Figure 2.8: C and C++ Logo

The C and C++ programming languages represent two of the most influential and widely used tools in the history of computer science. Known for their performance, versatility, and low-level control, they are employed in a vast range of sectors, from operating systems to high-performance applications.

Below is a brief overview of both languages:

**The C Language**:

- Developed in the 1970s by Dennis Ritchie, it was one of the first high-level languages in history.
- it is renowned for its simplicity, efficiency, and direct access to memory, making it ideal for writing highly optimized code [6].

**The C++ Language**:

- C++ is an extension of C developed in the 1980s by Bjarne Stroustrup. Among its key features is the support for Object-Oriented Programming (OOP), which allows for the creation of more structured and modular software compared to procedural C [14].
- It provides a rich Standard Library (STL) and is the industry standard for performance-critical applications.

In the context of this project, C plays a critical architectural role. It was utilized to define the Application Binary Interface (ABI) between the scheduler and the various hardware wrappers. Instead C++ serves as the primary development language. Its adoption was mandatory as it is the native language required by the SDKs of all the utilized frameworks, It is used to implement the complex logic of the scheduler and the data structures necessary for task management. Further details on this will be provided in the Implementation Chapter 4.

## 2.7 Cmake

Figure 2.9: Cmake Logo

To manage the compilation process of such a complex architecture, the project utilizes CMake (Cross-Platform Make). CMake is an open-source family of tools designed to build, test, and package software. It was originally created by Kitware in 2000 to address the need for a powerful build environment capable of supporting cross-platform development [7].

CMake does not build the software directly, instead it acts as a generator: it reads configuration files named CMakeLists.txt, and generates standard build files for the native toolchain of the operating system, such as Makefiles for Linux or Visual Studio projects for Windows. This abstraction allows developers to describe how the project should be built in a way that is independent of the specific compiler being used.

In this thesis its primary function is to manage the complexity of dynamic linking: since the project depends on large external frameworks (CUDA, OpenVINO, OneAPI, OpenBlass), CMake is configured to locate and link these as shared libraries. This ensures that the executable remains lightweight and that dependencies are resolved correctly at runtime.

Additionally, it enables a modular structure where specific hardware backends can be included or excluded via simple configuration flags.

# Chapter 3

# Logical Design

This chapter describes the logical structure of the project, the design process was divided into two main phases:

- The first part of the chapter (3.1) presents the Independent Benchmarks. Before developing the centralized scheduler, standalone applications were created for each accelerator. This phase was essential to verify the feasibility of the project and to understand how to effectively control the specific hardware using the selected libraries.
- The second part (3.2) provides a comprehensive overview of the AMM Scheduler. This section describes the high-level operation of all the components of the scheduler, it illustrates the architectural design of the main project and explains how the various logical blocks interact to enable the scheduler's functionality, detailing the specific operational logic that governs the system.

## 3.1 Preliminary Phase

Before designing the centralized scheduler, the project required a preliminary phase focused on the independent analysis of each software stack. Three standalone applications were developed, targeting the NVIDIA GPU, the Intel iGPU, and the NPU respectively. The primary objective of this phase to gain a deep understanding of the specific libraries (CUDA, SYCL, and OpenVINO) and their interaction with the hardware.

It is important to note that the OpenBLAS backend for the CPU was not included in this preliminary phase. This component was introduced directly during the development of the scheduler (described in the next section 3.2) with a specific goal: to maximize the degree of heterogeneity of the system, by treating the CPU as an additional accelerator rather than just a coordinator. Moreover unlike the GPU and NPU, which required a specific study of their asynchronous drivers and memory models, the CPU integration relies on standard synchronous function calls, consequently, a dedicated standalone analysis deemed unnecessary. The technical details of this integration will be discussed in the implementation chapter (4).

### 3.1.1 Studying the Common Patterns

A crucial goal of this stage was to gain a deep, hands-on understanding of the specific libraries. Since technologies like CUDA, SYCL, and OpenVINO manage hardware in fundamentally different ways, so a first approach phase was necessary to see how their specific mechanisms works and the drivers interactions.

This distinction is particularly evident in the case of OpenVINO, while CUDA and SYCL follow a traditional *kernel launch* paradigm, the NPU is designed exclusively for deep learning graphs, not for general-purpose linear algebra, consequently, to perform a standard matrix multiplication (GEMM) on this accelerator, a workaround was required, instead of invoking a mathematical function, the operation had to be encapsulated within a synthetic neural network layer, effectively "tricking" the driver into treating a raw calculation that we wanted to accomplish as an inference task (more on this in chapter 4).

Despite these architectural differences, this preliminary study revealed a recurring pattern, the execution flow for every accelerator whether running a kernel or a simulated inference relies on four identical logical steps:

- **Context Initialization**: setting up the driver handles like context, queue, and so on.
- **Memory Management**: allocating buffers and managing host-to-device transfers.

- **Kernel Execution**: triggering the asynchronous computation.
- **Synchronization**: waiting for the hardware to complete the task.

Recognizing this common structure for each library was fundamental for the design of the Abstraction Layer used in the scheduler.

### 3.1.2 Benchmarking

During this phase, benchmarking routines were implemented within each of the standalone applications. However, their purpose was distinct from the final experimental results presented in later chapters, in this context measuring execution time and power consumption served as a development tool to:

- Verify the correctness of the implementation (e.g., ensuring the code was actually running on the accelerator and not falling back to the CPU).
- Analyze the runtime behavior of the drivers, and identify architectural bottlenecks (such as PCIe transfer overheads or compilation latencies).

However, it must be noted that the performance profile observed in these isolated environments proved to be different from the behavior within the full scheduler. The concurrent execution of different workloads and the increased stress on system resources exposed new bottlenecks that were not visible during the standalone tests, consequently, the backend implementations were significantly evolved during the integration phase, and specific structures were introduced to mitigate these latencies. These optimizations and the specific issues encountered under high-load scenarios will be analyzed in detail in the Implementation Chapter (4).

## 3.2 AMM Scheduler

The core of the project is the AMM Scheduler, that stands for Asymettric Matrix Multiplication Scheduler.

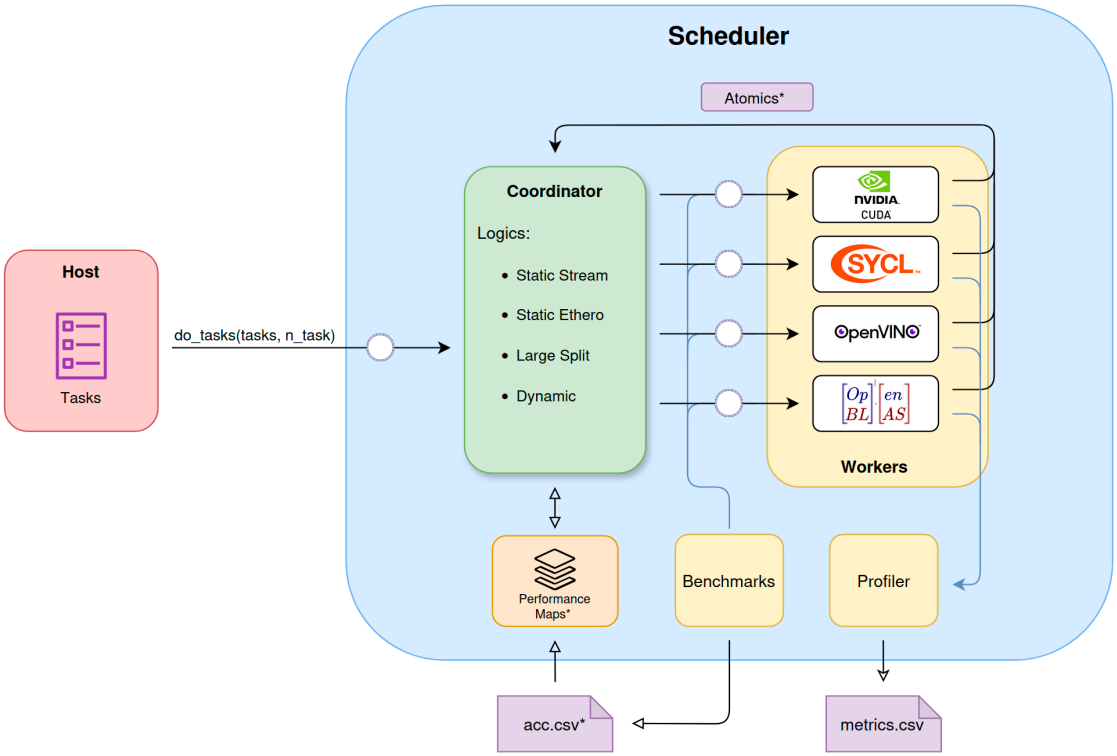


Figure 3.1: AMM Scheduler Components

As illustrated in figure 3.1, the architecture implements the producer and consumer paradigm.

In this model, the host application acts as the producer, generating computational tasks for the scheduler, that functions as the consumer, processing these requests asynchronously. The execution logic follows the farm pattern: a central coordinator retrieves the tasks and distributes them among the available worker nodes. To enable parallel execution, the coordinator and each worker operate as independent threads.

As shown in the diagram, Workers communicate with the Coordinator using atomic variables. This mechanism signals when a Worker is idle or updates the count of completed tasks, this ensures that the scheduler's wait function can correctly determine when all tasks are finished.

Now we will examine each component in the diagram and explain a bit deeper its function:

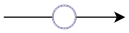

Figure 3.2: Ring Buffer Rappresentation

- **Ring Buffers**: The key to this asynchronous pattern is the ring buffer (rappresented in the figure 3.3 as the arrows marked with a circle), which decouples the Host from the Coordinator, designed ad-hoc for the scheduler, this structure enables extremely fast communication with minimal overhead, using atomic variables instead of locks. As the graph illustrates, this mechanism also serve the communication between the Coordinator and the Workers, minimizing overhead and maximizing speed during task distribution. More on the code implementation in chapter **??**.


Figure 3.3: Coordinator Rappresentation

- **Coordinator**: The Coordinator acts as the brain of the scheduler, it decides how to dispatch tasks and implements different strategies to demonstrate various resource management techniques.

  It features distinct scheduling logics, based on the type of workload that the host have to compute, the main logics that shape the design of the scheduler are the static one, the first, Static stream, handles static homogeneous workloads (tasks of identical matrices). By using a Performance Map (a custom data structure containing heuristics and execution time estimates) the coordinator predicts how long a task will take on a specific accelerator and based on this, it performs a static dispatch of the tasks in a batch style (see section static partitioning **??**).

  The second logic, Static Hetero, work in a similar way but for static heterogeneous workloads (matrices of different sizes), the coordinator uses the same performance map to calculate the optimal distribution, ensuring the workload is balanced across all accelerators (see section static hetero partitioning **??**).

To evaluate these strategies, we also implemented a Dynamic logicl This approach assigns tasks to the first available Worker (idle state) and serves as a baseline to verify the efficiency of our static methods.

Finally, the Coordinator supports Task Splitting, if a matrix exceeds the memory capacity of a single accelerator, the logic breaks it down into smaller subtasks to be distributed across multiple accellerator.

- **Workers**:
- **Benchmarks and Performance Maps**:
- **Profiler**:

# Chapter 4

# Implementation

# Chapter 5

# Results

# Appendix A

# Questa è un'appendice

Fusce mauris. Vestibulum luctus nibh at lectus. Sed bibendum, nulla a faucibus semper, leo velit ultricies tellus, ac venenatis arcu wisi vel nisl. Vestibulum diam. Aliquam pellentesque, augue quis sagittis posuere, turpis lacus congue quam, in hendrerit risus eros eget felis. Maecenas eget erat in sapien mattis porttitor. Vestibulum porttitor. Nulla facilisi. Sed a turpis eu lacus commodo facilisis. Morbi fringilla, wisi in dignissim interdum, justo lectus sagittis dui, et vehicula libero dui cursus dui. Mauris tempor ligula sed lacus. Duis cursus enim ut augue. Cras ac magna. Cras nulla. Nulla egestas. Curabitur a leo. Quisque egestas wisi eget nunc. Nam feugiat lacus vel est. Curabitur consectetuer.

# Bibliography


[1] Intel Corporation. *Intel Arc Graphics Xe-LPG Architecture*, 2023. https://www.intel.com/content/www/us/en/docs/oneapi/optimization-guide-gpu/2023-2/intel-xe-gpu-architecture.html.

[2] Intel Corporation. *Intel Core Ultra Processors (Series 1)*, 2024. https://www.intel.com/content/www/us/en/products/sku/236849/intel-core-ultra-9-processor-185h-24m-cache-up-to-5-10-ghz/specifications.html.

[3] Intel Corporation. Intel oneapi: A unified programming model, 2024. https://www.intel.com/content/www/us/en/developer/tools/oneapi/overview.html.

[4] Intel Corporation. *Intel's Neural Processing Unit*, 2024.

[5] Intel Corporation. *Openvino Documentation*, 2025. https://docs.openvino.ai/2025/index.htm.

[6] Brian Kernighan and Dennis Ritchie. *The C Programming Language*. Prentice Hall, 1988.

[7] Kitware, Inc. *CMake Documentation*, 2025. https://cmake.org/cmake/help/latest/.

[8] Klaus Hinum. *NVIDIA GeForce RTX 4060 Laptop GPU Spec*, 2022. https://www.notebookcheck.net/NVIDIA-GeForce-RTX-4060-Laptop-GPU-Benchmarks-and-Specs.675692.0.html.

[9] NVIDIA Corporation. *Cublas*, 2025. https://docs.nvidia.com/cuda/cublas/index.html.

[10] NVIDIA Corporation. *CUDA C++ Programming Guide*, 2025. https://docs.nvidia.com/cuda/cuda-c-programming-guide/.

[11] NVIDIA Corporation. *NVIDIA Management Library (NVML)*, 2026. https://developer.nvidia.com/management-library-nvml.

[12] OpenBLAS Team. Openblas github, 2024. https://github.com/OpenMathLib/OpenBLAS.

[13] Steve Rennich. *CUDA C++ Streams and Concurrency*, 2024. https://developer.download.nvidia.com/CUDA/training/StreamsAndConcurrencyWebinar.pdf.

[14] Bjarne Stroustrup. *The C++ Programming Language*. Addison-Wesley, 2013.

[15] The Khronos Group. Sycl overview and specification, 2024. https://www.khronos.org/sycl/.

[16] Zhang Xianyi. Openblas, 2025. http://www.openmathlib.org/OpenBLAS/.